

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_24

24. Visitor Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Visitor pattern.

GoF Definition

It represents an operation to be performed on the elements of an object structure. The Visitor pattern lets you define a new operation without changing the classes of the elements on which it operates.

Concept

This pattern can be used in many different scenarios. Using this pattern, you often add new operations to an object without modifying its class definition. To achieve this, you separate an algorithm from the object structure. This may sound surprising to you, but it is one of the key usages of this pattern. This pattern also promotes the Open/Closed Principle.

Note This is another design pattern that may seem difficult to understand at the beginning. I'll make it simple for you by excluding the "object structure" in demonstration 1. But once you understand this implementation, it will be easier for you to understand demonstration 2 where you can experience the true power of this design pattern. In demonstration 2, I combine it with the Composite pattern. Q&A 24.8 discusses the concept of the object structure in brief.

You can think of an object structure as a container. It can be helpful to think of it like a tree structure where different objects (nodes) are arranged together to form the tree. A visitor object traverses each node in the tree and captures its state. Once the visitor captures the state, you call the visitor-specific method. You'll learn this in detail through the different implementations in this chapter. But note a simple fact: if you deal with only one element, there is no need to create an object structure. This is why I made demonstration 1 for you.

To understand this pattern, let's consider a simple scenario. Suppose you have a tree structure where each node in the tree has methods that work differently. But your customer wants you to add more methods or update these methods (e.g., the top management of an organization changes the “promotion criteria” or the “appraisal process” frequently). You can accept this customer request as a one-off, but if your client passes a similar request multiple times and very often, will it be possible for you to introduce methods like this in each possible case, particularly when the overall code structure is very big and complex? Also, in a tree structure, if it is a branch node, can you imagine the impact of these changes across other nodes?

So, you may think of some way to handle your fickle-minded customers. The Visitor pattern can help you in a situation like this. You'll see such an implementation in demonstration 1.

Real-Life Example

Think of a taxi-booking scenario. When the taxi arrives at your door and you enter the vehicle, the taxi takes control of the transportation. It can take you to your destination through a new route that you are not familiar with, and in the worst case, it can alter the destination (which is a case generated due to improper use of the Visitor pattern).

Computer World Example

This pattern is useful when public APIs need to support *plug-in* operations. Clients can then perform their intended operations on a class (with the visiting class) without modifying the source. Here is another example for you:

- When you use the interface `javax.lang.model.element.ElementVisitor<R, P>` (where R is the return type of the visitor's method and P is the type of the additional parameter to the visitor's method), you follow this pattern. The JDK documentation says the following about it:

“A visitor of program elements, in the style of the visitor design pattern. Classes implementing this interface are used to operate on an element when the kind of element is unknown at compile time. When a visitor is passed to an element's accept method, the `visitXYZ` method most applicable to that element is invoked. ...”

Implementation

I told you that you will see different implementations of the Visitor pattern in this chapter. Let's start with an example that follows the core theory of this pattern. In the next example, I enhance the example to demonstrate the real power of this pattern.

In common implementations of the Visitor pattern, you see the presence of an "object structure with multiple nodes." In such cases, remember the key steps, which are summarized as follows:

- There are two inheritance hierarchies: one is used for the existing objects hierarchy and the other one is used for the visitor hierarchy.
- A visitor visits each object one by one in this object structure
- When the visitor visits an object, that object invokes a specific method defined in the visitor.
- During this invocation, the object passes itself as an argument.
- In each object, there is an `accept` method. This method accepts the visitor as an argument. It helps an object invoke the particular method of the visitor. Here is a general format for this (the important portions are in bold):

```
public void acceptMethod(Visitor visitor){  
    visitor.specificVisitorMethod(this);  
}
```

- When you need a new operation/functionality, you introduce another implementor of the visitor interface and implement the operation as per your need.

In demonstration 1, the concrete class `IntegerProcessor` implements the interface `NumberProcessor`. Inside the `IntegerProcessor` class is an integer called `number`. When you create an instance of this class, it is initialized with a value, say 5.

Now suppose you want to update this initialized value and display it. You can do it in different ways. For example, you can add a setter method inside `IntegerProcessor` to perform your job, or you can use the Visitor pattern, which I am about to explain.

Let's assume you opt for a setter method that may look something like the following:

```
public void setNumber(int newValue){  
    number=newValue;  
}
```

```
}
```

But this approach will not work if you tag the `final` keyword with the number. We often use the `final` keyword to promote immutability. So, if you need to update a fixed number and use it as a variable, you need to write some intelligent code. *In a case like this, you may want to use the Visitor pattern where you do not alter the actual number, but modify the associated value based on your need.* To demonstrate a simple case study, in the following implementation, you increase the number by adding 10 to it using the Visitor design pattern.

The first and foremost thing for you to notice is the presence of the following method inside the `IntegerProcessor` class :

```
public void acceptVisitor(Visitor visitor){
    visitor.visitNumber(this);
}
```

It gives you a clue that this method allows a `Visitor` type to do some intended work. It also indicates that there must be a method named `visitNumber(...)` inside the `Visitor` type. Let's see the `Visitor` hierarchy now:

```
interface Visitor {
    // The method to visit the IntegerProcessor.
    void visitNumber(IntegerProcessor myInt);
}
class IncrementorVisitor implements Visitor {

    @Override
    public void visitNumber(IntegerProcessor myInt) {
        System.out.println("The flag value is:" + myInt.getNumber());
        System.out.println("Incrementing it by 10.");
        int temp=myInt.getNumber()+10;
        System.out.println("The new value is:" + temp);
        // Remaining code, if any
    }
}
```

Notice that the `visitNumber(...)` method accepts an `IntegerProcessor` argument and does some work to fulfil its need. Here you increase the number by 10. This is why this code snippet in the client code

```
NumberProcessor targetInteger = new IntegerProcessor();
Visitor visitor = new IncrementorVisitor();
```

```
targetInteger.acceptVisitor(visitor);
```

can produce the following output:

```
The flag value is:5
Incrementing it by 10.
The new value is:15
```

What is the benefit of using this approach? You'll understand it better once you cover all of the demonstrations in this chapter. For now, the first thing to notice is in the current situation, if you need one more different operation that can double the number, you may use another visitor, say `DoubleMakerVisitor`, and you can write something like the following:

```
class DoubleMakerVisitor implements Visitor {
    @Override
    public void visitNumber(IntegerProcessor myInt) {
        System.out.println("The flag value is:" + myInt.getNumber());
        System.out.println("Multiplying it by 2.");
        int temp=myInt.getNumber() *2;
        System.out.println("The new value is:" + temp);
        // Remaining code, if any
    }
}
```

And once you append the following two more lines (shown in bold) in the client code as follows

```
NumberProcessor targetInteger = new IntegerProcessor();
Visitor visitor = new IncrementorVisitor();
targetInteger.acceptVisitor(visitor);
visitor = new DoubleMakerVisitor();
targetInteger.acceptVisitor(visitor);
```

you get the following output:

```
The flag value is:5
Incrementing it by 10.
The new value is:15
The flag value is:5
Multiplying it by 2.
The new value is:10
```

Now you can see an advantage: when you include the operation to double the number, you do not add the behavior inside the `NumberProcessor` hi-

erarchy . Notice that in the first case, you add 10 to a number and later, you make a double of the number. But inside the `IntegerProcessor` class, you did not add an additional method when you added the new behavior.

Similarly, if you need any more operations, you can add the intended operation in the `Visitor` hierarchy instead of adding it to the `NumberProcessor` hierarchy. As a result, you get a greater benefit: you do not need to alter existing code (the `NumberProcessor` hierarchy in this example) and thus you can avoid retesting it.

Now look at the complete demonstration before we further discuss the pros and cons of this approach.

Class Diagram

Figure 24-1 shows the important parts of the class diagram.

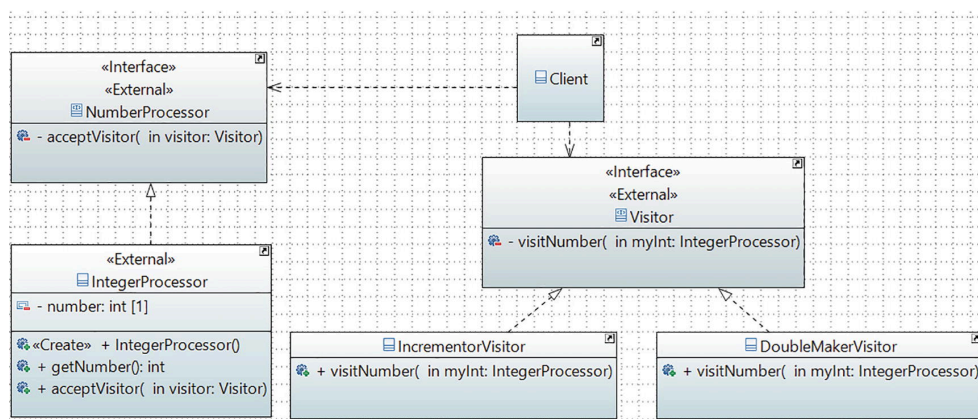


Figure 24-1 Class diagram

Package Explorer View

Figure 24-2 shows the high-level structure of the program.

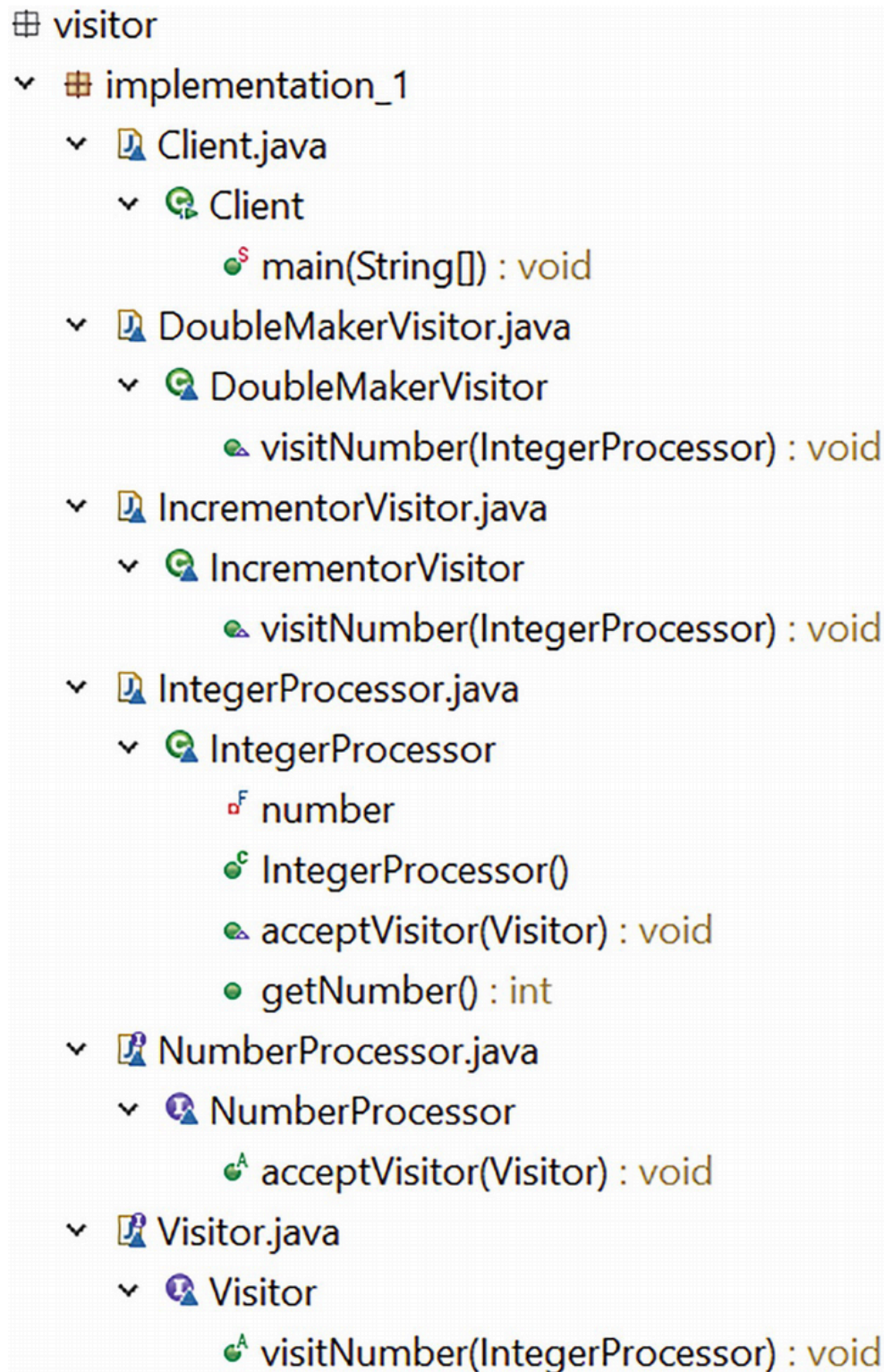


Figure 24-2 Package Explorer view

Demonstration 1

Here's the complete demonstration. All parts are stored inside the package named `jdp3e.visitor.implementation_1`.

```
// NumberProcessor.java
interface NumberProcessor {
    void acceptVisitor(Visitor visitor);
}
```

```

// IntegerProcessor.java
class IntegerProcessor implements NumberProcessor {
    // The "flag" is final. Once initialized,
    // it should not be changed.
    private final int number;
    public IntegerProcessor(){
        number=5;//Initial or default value
    }
    public int getNumber(){
        return number;
    }

    @Override
    public void acceptVisitor(Visitor visitor){
        visitor.visitNumber(this);
    }
}

// Visitor.java
interface Visitor {
    //The method to visit the IntegerProcessor.
    void visitNumber(IntegerProcessor myInt);
}

// IncrementorVisitor.java
class IncrementorVisitor implements Visitor {
    @Override
    public void visitNumber(IntegerProcessor myInt) {
        System.out.println("The flag value is:"+
            myInt.getNumber());
        System.out.println("Incrementing it by 10.");
        int temp=myInt.getNumber()+10;
        System.out.println("The new value is:"+ temp);
        // Remaining code, if any
    }
}

// DoubleMakerVisitor.java
class DoubleMakerVisitor implements Visitor {
    @Override
    public void visitNumber(IntegerProcessor myInt) {

        System.out.println("The flag value is:"+ myInt.getNumber());
        System.out.println("Multiplying it by 2.");
        int temp=myInt.getNumber() *2;
        System.out.println("The new value is:"+ temp);
        // Remaining code, if any
    }
}

```



```
}  
// Client.java  
class Client {  
    public static void main(String[] args) {  
        System.out.println("***Visitor Pattern Demonstration-1***\n");  
        NumberProcessor targetInteger = new  
            IntegerProcessor();  
        System.out.println("Testing the IncrementorVisitor now.");  
        Visitor visitor = new IncrementorVisitor();  
        targetInteger.acceptVisitor(visitor);  
        System.out.println("\nTesting the DoubleMakerVisitor now.");  
        visitor = new DoubleMakerVisitor();  
        targetInteger.acceptVisitor(visitor);  
    }  
}
```

Output

Here's the output:

```
***Visitor Pattern Demonstration-1***  
Testing the IncrementorVisitor now.  
The flag value is:5  
  
Incrementing it by 10.  
The new value is:15  
Testing the DoubleMakerVisitor now.  
The flag value is:5  
  
Multiplying it by 2.  
The new value is:10
```

Analysis

Notice that to achieve the goal, in the previous example, you separate the functionality implementations (i.e., algorithms) from the original class hierarchy. This approach helps you add the new functionality without altering the existing code (the `NumberProcessor` hierarchy) and thus avoid retesting it.

Using Visitor Pattern and Composite Pattern Together

In Demonstration 1, you saw a simple example of the Visitor design. Next is another implementation, but this time it's combined with the Composite pattern.

Let's consider the example of the Composite design pattern from Chapter 13. In that example, there is a college with two different departments. Each department has one head of department (HOD) and multiple teachers/lecturers. All of these HODs report to the principal of the college. I won't explain the college structure in detail again, because it is the same as described in Chapter 13.

- The Mathematics teachers are M. Jacob and M. Rustom. The Computer Science and Engineering (CSE) teachers are C. Kate, C. Jones, and C. Proctor. These lecturers do not supervise anyone, so they are treated as leaf nodes in the following tree diagrams.
- S. Som is the Principal of the college, and he holds the highest position. Two HODs named S. Das (the HOD of Mathematics) and V. Sarcar (the HOD of CSE) report to the Principal. The HODs and the Principal are non-leaf nodes.

Figure 24-3 shows the sample tree diagram for your immediate reference.

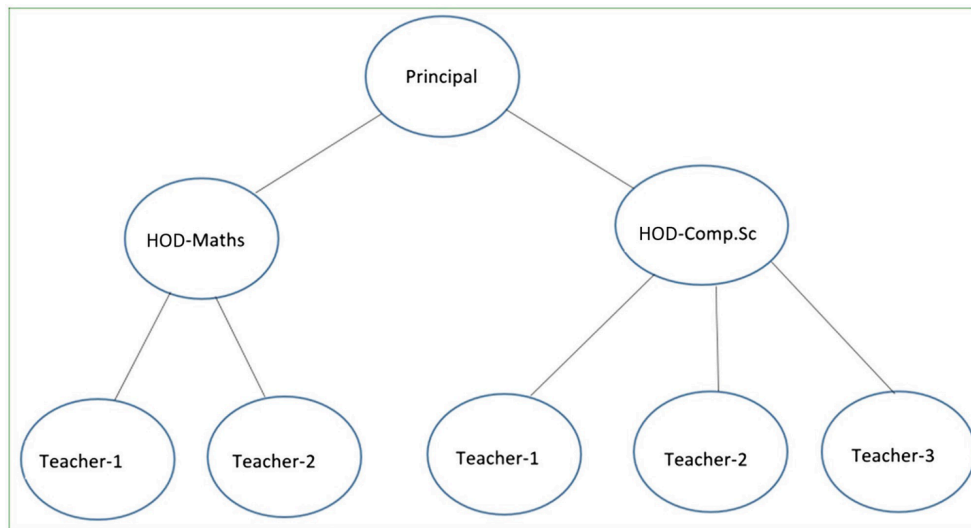


Figure 24-3 Tree structure taken from Chapter 13 (Figure 13-1)

Now suppose, as per the recommendation of higher management, the principal of the college wants to promote some employees. Let's say that teaching experience is the only criteria to promote someone, but the criteria vary among senior teachers (branch nodes) and junior teachers (leaf nodes) as follows: for a junior teacher, the minimum

criterion for promotion is 5 years, and for senior teachers, it is 15 years.

You understand that the promotion criterion may change in the future and there may be more requirements from the higher authorities. So, the Visitor pattern is a perfect fit to fulfill the current requirement. This is why in the upcoming example, you add a new attribute and a new method in the abstract class `Employee` as follows:

```
// New field for this example.  
// It is tagged with "final", so a visitor cannot modify it.  
protected final double experience;  
// Newly added for this example  
public abstract void acceptVisitor(Visitor visitor);
```

You understand that the concrete implementors of the `Employee` class (`SeniorEmployee` and `JuniorEmployee`) need to override this method as per this requirement.

In addition, the getter methods are also present in the `Employee` class:

```
// Gets the years of experience  
public double getExperience() {  
    return experience;  
}  
// Get the name of employee  
public String getName() {  
    return name;  
}
```

Following the design in demonstration 1, you call the visitor interface `Visitor`. This interface has two overloaded methods as follows:

```
interface Visitor {  
    void visitTheElement(SeniorEmployee employees);  
    void visitTheElement(JuniorEmployee employee);  
}
```

The promotion criteria for a senior employee and a junior employee are different. So, you see the following code (refer to the comments for easy understanding):

```
class PromotionCheckerVisitor implements Visitor {  
    @Override  
    public void visitTheElement(SeniorEmployee employee) {  
        // We'll promote them if experience is  
        // greater than 15 years
```

```

        boolean eligibleForPromotion =
            employee.getExperience() > 15 ? true : false;
        System.out.println("\t\t" + employee.getName() +
            " is eligible for promotion? " +
            eligibleForPromotion);
    }
    @Override
    public void visitTheElement(JuniorEmployee employee) {
        // We'll promote them if experience is
        // greater than 5 years
        boolean eligibleForPromotion =
            employee.getExperience() > 5 ? true : false;
        System.out.println("\t\t" + employee.getName() + "
            is eligible for promotion? " +
            eligibleForPromotion);
    }
}

```

Demonstration 2 has a container (a List data structure called `participants`) in the client code. When a visitor gathers the necessary details from this college structure, it can show the eligible candidates for promotion. This is the reason to include the following code segment:

```

private static void checkPromotionEligibility(List<Employee> container) {
    Visitor visitor = new PromotionCheckerVisitor();
    // Principal holds the highest position.
    // He does not need a promotion.
    System.out.println("\nChecking the eligible
        candidates for a promotion.");
    for (Employee emp : container) {
        emp.acceptVisitor(visitor);
    }
}

```

Notice that the visitor collects the data one by one from the original college structure without making any modifications to it. It analyzes the data before presenting the final result. To understand this visually, you can follow the arrows in the upcoming figures. *The principal is at the top of the organization; so, assume that at present no promotion is required for him. (Do not worry! In demonstration 3, we'll promote him too).*

Step 1

Figure [24-4](#) shows step 1.

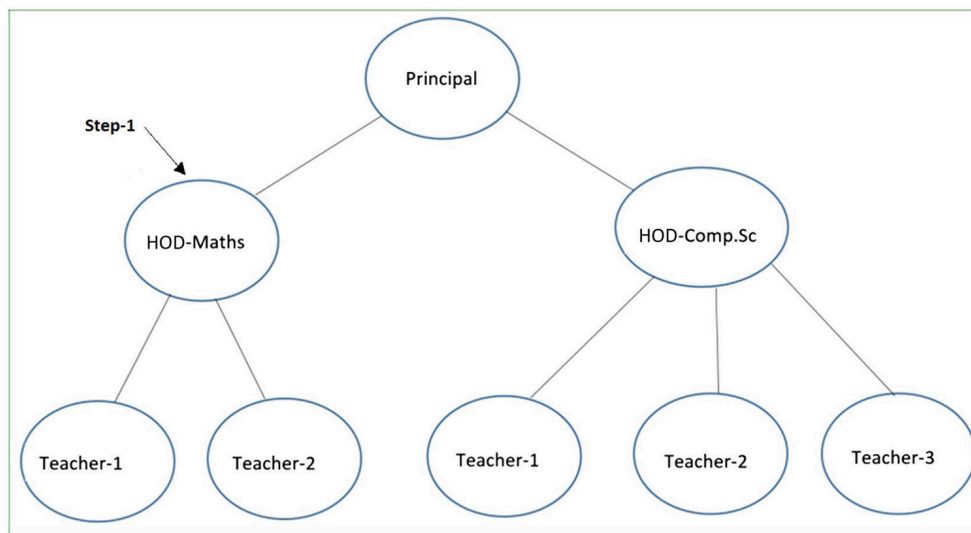


Figure 24-4 Step 1

Step 2

Figure 24-5 shows step 2.

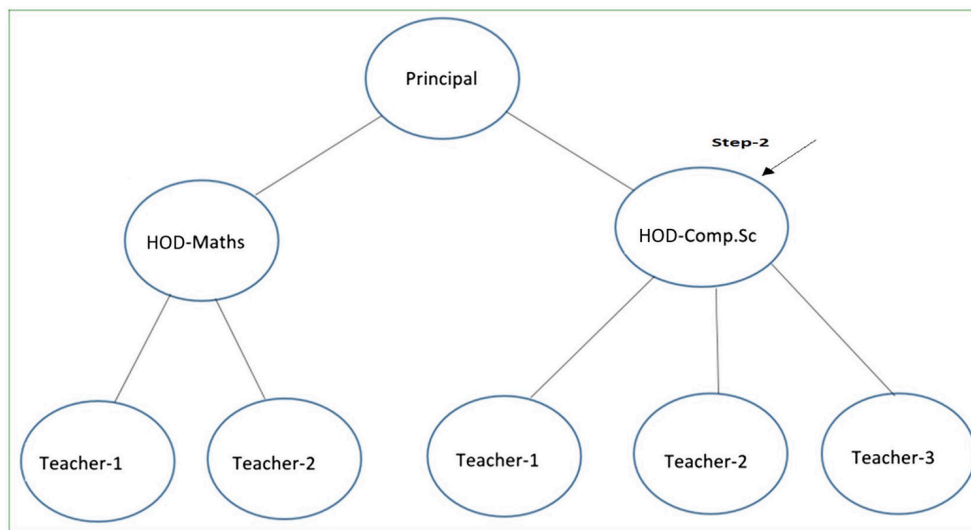


Figure 24-5 Step 2

Step 3

Figure 24-6 shows step 3.

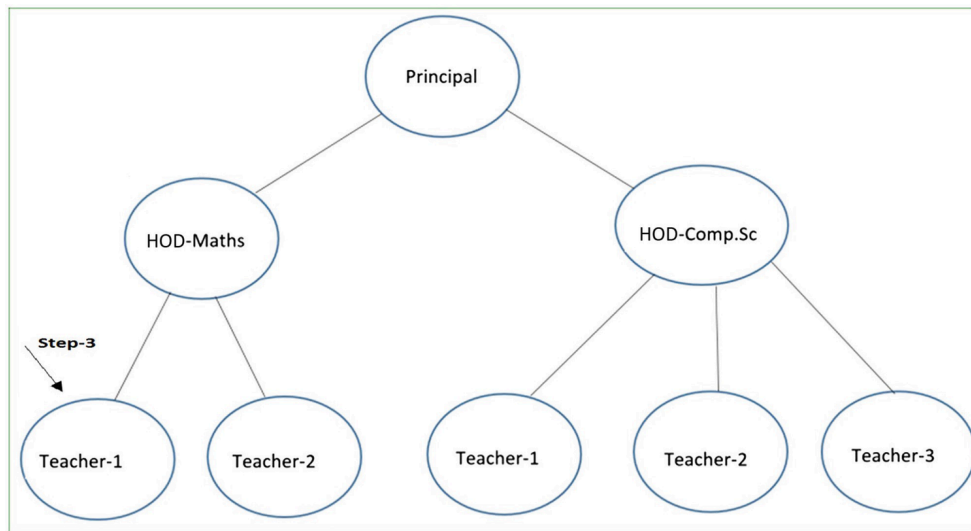


Figure 24-6 Step 3

Step 4

Figure 24-7 shows step 4.

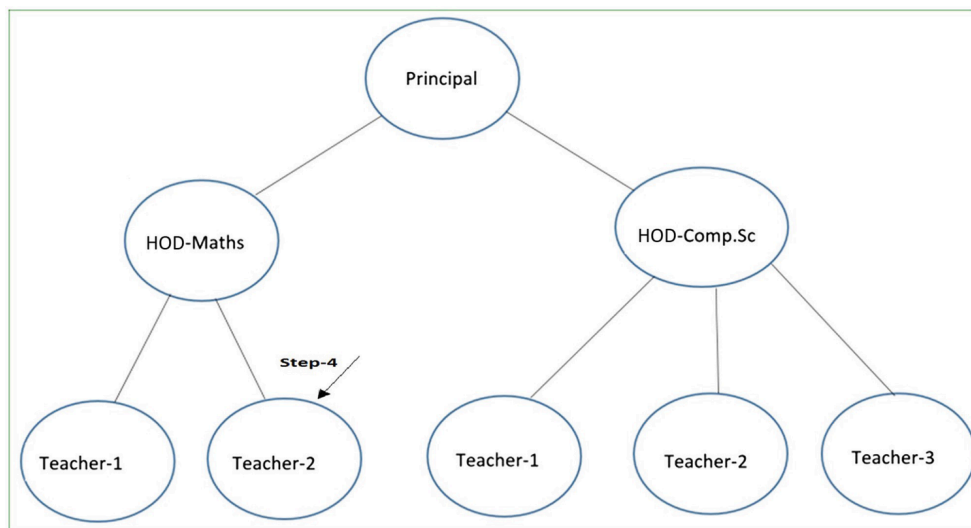


Figure 24-7 Step 4

Step 5

Figure 24-8 shows step 5.

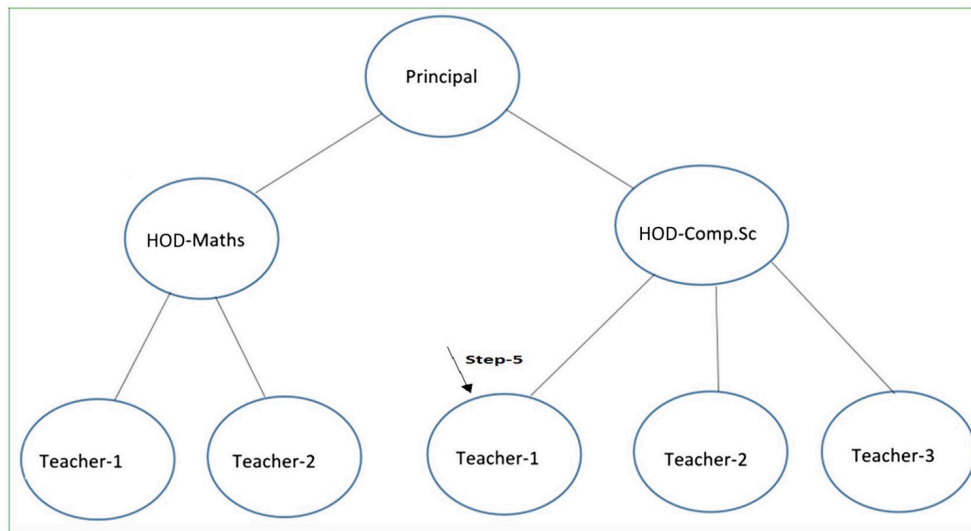


Figure 24-8 Step 5

And so on...

NoteI followed the design from Demonstration 1 in this chapter and the code example is built on top of the only demonstration in Chapter [13](#). I restructured the client code so that you can understand it easily. I removed unnecessary segments that are not relevant to this discussion. For brevity, I do not include the class diagram and Package Explorer view separately for this example.

So, let's jump to the following demonstration.

Demonstration 2

Here's the implementation. All parts are stored inside the package

`jdp3e.visitor.implementation_2`.

```

import java.util.ArrayList;
import java.util.List;
// Employee.java
import java.util.ArrayList;
import java.util.List;
abstract class Employee {
    protected String name;
    protected String designation;
    protected int subordinatesCount;
    protected List<Employee> subordinates;
    // New field for this example.
    // It is tagged with "final", so a visitor
    // cannot modify it.
    protected final double experience;
    // Constructor
    public Employee(String name, String designation, double experience) {

```



```

        this.name = name;
        this.designation = designation;
        this.subordinatesCount = 0;
        this.experience = experience;
        subordinates = new ArrayList<Employee>();
    }
    protected abstract void printStructures();
    protected abstract int getSubordinatesCount();
    protected abstract void addEmployee(Employee e);
    protected abstract void removeEmployee(Employee e);
    // Gets the years of experience
    public double getExperience() {
        return experience;
    }
    // Get the name of employee
    public String getName() {

        return name;
    }
    public abstract void acceptVisitor(Visitor visitor);
}
// SeniorEmployee.java
// It is a non-leaf node
class SeniorEmployee extends Employee {
    public SeniorEmployee(String name, String designation,
                           double experience) {
        super(name, designation, experience);
    }
    @Override
    protected void printStructures() {

        System.out.println("\t" + this.name + " is a " +
                             this.designation + ". Experience:" +
                             this.experience);
        for (Employee e : subordinates) {
            e.printStructures();
        }
    }
    @Override
    protected int getSubordinatesCount() {
        subordinatesCount = subordinates.size();
        for (Employee e : subordinates) {
            subordinatesCount += e.getSubordinatesCount();
        }
        return subordinatesCount;
    }
    @Override

```

```

        protected void addEmployee(Employee e) {
            subordinates.add(e);
        }
        @Override
        protected void removeEmployee(Employee e) {
            subordinates.remove(e);
        }
        @Override
        public void acceptVisitor(Visitor visitor) {

            visitor.visitTheElement(this);
        }
    }
}

// JuniorEmployee.java
class JuniorEmployee extends Employee {
    public JuniorEmployee(String name, String designation,
                           double experience) {
        super(name, designation, experience);
    }
    @Override
    protected void printStructures() {

        System.out.println("\t\t" + this.name + " is a " +
                            this.designation + ". Experience: " +
                            this.experience);
    }
    @Override
    protected int getSubordinatesCount() {
        return subordinatesCount; // Always 0
    }
    @Override
    protected void addEmployee(Employee e) {
        // Empty method
    }
    @Override
    protected void removeEmployee(Employee e) {
        // Empty method
    }
    @Override
    public void acceptVisitor(Visitor visitor) {
        visitor.visitTheElement(this);
    }
}

// Visitor.java

interface Visitor {

```

```

void visitTheElement(SeniorEmployee employees);
void visitTheElement(JuniorEmployee employee);
//void visitTheElement(Employee employee);
}

// PromotionCheckerVisitor.java
class PromotionCheckerVisitor implements Visitor {
    @Override
    public void visitTheElement(SeniorEmployee employee) {
        // We'll promote them if experience is

        // greater than 15 years
        boolean eligibleForPromotion =
            employee.getExperience() > 15 ? true : false;
        System.out.println("\t\t" + employee.getName() + "
                                is eligible for promotion? " +
                                eligibleForPromotion);
    }
    @Override
    public void visitTheElement(JuniorEmployee employee) {
        // We'll promote them if experience is
        // greater than 5 years
        boolean eligibleForPromotion =
            employee.getExperience() > 5 ? true : false;
        System.out.println("\t\t" + employee.getName() + "
                                is eligible for promotion? " +
                                eligibleForPromotion);
    }
}

// Client.java
import java.util.ArrayList;
import java.util.List;
class Client {
    public static void main(String[] args) {
        System.out.println("***Composite Pattern and
                                Visitor Pattern Demonstration***");
        Employee hodMath = formMathDept();
        Employee hodCompSc = formComputerScDept();
        Employee principal = formPrincipal(hodMath, hodCompSc);
        printCollegeStructure(principal);
        List<Employee> participants =
            createContainer(principal, hodMath, hodCompSc);
        checkPromotionEligibility(participants);
    }

    private static void printCollegeStructure(Employee principal) {
        // Printing the leaf-nodes and branches
        // in the same way i.e. in each case, we

```

```

        // are calling printStructures() method.
        System.out.println("The details of the Principal object:");
        // Prints the complete college structure
        principal.printStructures();
    }

    private static Employee formComputerScDept() {
        // Three lecturers other than HOD work
        // in the Computer Science Department.
        Employee cseTeacher1 = new JuniorEmployee(
            "C.Kate", "Lecturer", 8.2);
        Employee cseTeacher2 = new JuniorEmployee(
            "C.Jones", "Lecturer", 4.5);
        Employee cseTeacher3 = new JuniorEmployee(
            "C.Proctor", "Lecturer", 6);
        Employee hodCompSc = new SeniorEmployee(
            "Mr. V.Sarcar", "HOD(CSE)", 15.5);
        // Teachers of Computer Science
        // directly reports to HOD-CSE

        hodCompSc.addEmployee(cseTeacher1);
        hodCompSc.addEmployee(cseTeacher2);
        hodCompSc.addEmployee(cseTeacher3);
        return hodCompSc;
    }

    private static Employee formMathDept() {
        // Two lecturers other than HOD work
        // in the Mathematics department.
        Employee mathTeacher1 = new JuniorEmployee(
            "M.Jacob", "Lecturer", 7.5);
        Employee mathTeacher2 = new JuniorEmployee(
            "M.Rustom", "Lecturer", 3);
        Employee hodMath = new SeniorEmployee(
            "Mrs.S.Das", "HOD(Math)", 12.5);
        // Teachers of Mathematics directly
        // reports to HOD-Mathematics
        hodMath.addEmployee(mathTeacher1);
        hodMath.addEmployee(mathTeacher2);
        return hodMath;
    }

    private static Employee formPrincipal(Employee hodMath,
        Employee hodCompSc) {
        // Principal of the college
        Employee principal = new SeniorEmployee(
            "Dr.S.Som", "Principal", 21);
        // The Principal is on top of the college.
        // The HOD(Mathematics) and HOD(Computer Sc.)
        // directly report to him.
        principal.addEmployee(hodMath);
    }

```

```

        principal.addEmployee(hodCompSc);
        return principal;
    }

    private static List<Employee> createContainer(Employee
        principal, Employee hodMath, Employee hodCompSc) {
        List<Employee> employeeContainer = new
            ArrayList<Employee>();

        // For employees who directly reports to Principal
        for (Employee e : principal.subordinates) {

            employeeContainer.add(e);
        }
        // For employees who directly reports to HOD(Math)
        for (Employee e : hodMath.subordinates) {
            employeeContainer.add(e);
        }
        // For employees who directly reports to
        // HOD(Comp.Sc)
        for (Employee e : hodCompSc.subordinates) {
            employeeContainer.add(e);
        }
        return employeeContainer;
    }

    private static void checkPromotionEligibility(
        List<Employee> container) {
        Visitor visitor = new PromotionCheckerVisitor();
        // Principal holds the highest position.
        // He does not need a promotion.
        System.out.println("\nChecking the eligible
            candidates for a promotion.");
        for (Employee emp : container) {
            emp.acceptVisitor(visitor);
        }
    }
}

```

Output

Here's the output. Some portions are in bold to show you that the visitor was able to complete the job successfully.

```

***Composite Pattern and Visitor Pattern Demonstration***
The details of the Principal object:
    Dr.S.Som is a Principal. Experience:21.0
    Mrs.S.Das is a HOD(Math). Experience:12.5
        M.Jacob is a Lecturer. Experience: 7.5
        M.Rustom is a Lecturer. Experience: 3.0

```

```

Mr. V.Sarcar is a HOD(CSE). Experience:15.5
C.Kate is a Lecturer. Experience: 8.2
C.Jones is a Lecturer. Experience: 4.5
C.Proctor is a Lecturer. Experience: 6.0
Checking the eligible candidates for a promotion.
Mrs.S.Das is eligible for promotion? false
Mr. V.Sarcar is eligible for promotion? true
M.Jacob is eligible for promotion? true
M.Rustom is eligible for promotion? false
C.Kate is eligible for promotion? true
C.Jones is eligible for promotion? false
C.Proctor is eligible for promotion? True

```

Analysis

In order to avoid modifying the original data, in the `Employee` class you see the getter methods only. It's a better practice in an example like this. But you must remember that this is optional for you.

Demonstration 3

The point of Demonstration 2 is to help you understand the usage of the Composite pattern and the Visitor pattern together. Since I made this demonstration on top of the demonstration from Chapter [13](#), I did not make big changes.

I hope you have the idea now! So, it's time to refactor the code. Notice that in demonstration 2, you made a container before you traversed it using a visitor. But if you know that you can promote the principal too (say you want him to be included on the board of directors) and you'll visit all nodes of the tree, you do not need to create such a container separately. In addition, notice that this time, instead of showing subordinate counts, the focus is to investigate the potential candidates who are eligible for promotions. So, you drop the `getSubordinatesCount()` method and introduce a new method called `getSubordinates()`. By default, this method returns an empty list.

A junior employee (leaf node) does not supervise anyone. So, the `JuniorEmployee` class does not need to override the default implementation. But a composite node will override this method and return the list of subordinates. So, in the modified implementation, inside the `SeniorEmployee` class, you see the following code segment:

```
@Override
```

```
public Collection<Employee> getSubordinates(){
    return subordinates;
}
```

In demonstration 2, the visitor traverses the nodes as per the node arrangements in the container. In this demonstration, during the traversal process, once the visitor sees a composite node, it traverses all nodes associated with him. The associated output will confirm this.

There is no change in the visitor hierarchy. So, the remaining code is easy to understand. Here is the complete demonstration (the key changes are in bold). All parts are stored inside the package `jdp3e.visitor.implementation_3`.

```
// Employee.java
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.List;
abstract class Employee {
    protected String name;
    protected String designation;
    protected List<Employee> subordinates;
    // New field for this example.
    // It is tagged with "final", so a visitor cannot
    // modify it.
    protected final double experience;
    // Constructor
    public Employee(String name, String designation, double experience) {
        this.name = name;
        this.designation = designation;
        this.experience = experience;
        subordinates = new ArrayList<Employee>();
    }
    protected abstract void printStructures();
    protected abstract void addEmployee(Employee e);
    protected abstract void removeEmployee(Employee e);
    // Gets the years of experience

    public double getExperience() {
        return experience;
    }
    // Get the name of employee
    public String getName() {
        return name;
    }
    public Collection<Employee> getSubordinates(){
        return Collections.emptyList();
    }
}
```

```

        public abstract void acceptVisitor(Visitor visitor);
    }

    // SeniorEmployee.java
    // It is a non-leaf node
    class SeniorEmployee extends Employee {
        public SeniorEmployee(String name, String designation,
                               double experience) {
            super(name, designation, experience);
        }

        @Override
        protected void printStructures() {
            System.out.println("\t" + this.name + " is a " +
                               this.designation + ". Experience:" +
                               this.experience);
            for (Employee e : subordinates) {
                e.printStructures();
            }
        }

        @Override
        public Collection<Employee> getSubordinates(){
            return subordinates;
        }

        @Override
        protected void addEmployee(Employee e) {
            subordinates.add(e);
        }

        @Override
        protected void removeEmployee(Employee e) {
            subordinates.remove(e);
        }

        @Override
        public void acceptVisitor(Visitor visitor) {
            visitor.visitTheElement(this);
        }
    }

    // JuniorEmployee.java
    class JuniorEmployee extends Employee {

        public JuniorEmployee(String name, String designation,
                               double experience) {
            super(name, designation, experience);
        }

        @Override
        protected void printStructures() {
            System.out.println("\t\t" + this.name + " is a " +
                               this.designation + ". Experience: " +

```



```

        this.experience);
    }
    @Override
    protected void addEmployee(Employee e) {
        // Empty method
    }
    @Override
    protected void removeEmployee(Employee e) {
        // Empty method
    }
    @Override
    public void acceptVisitor(Visitor visitor) {
        visitor.visitTheElement(this);
    }
}
// Visitor.java
interface Visitor {
    void visitTheElement(SeniorEmployee employees);
    void visitTheElement(JuniorEmployee employee);
}
// PromotionCheckerVisitor.java
class PromotionCheckerVisitor implements Visitor {
    @Override
    public void visitTheElement(SeniorEmployee employee) {
        // We'll promote them if experience is greater

        // than 15 years
        boolean eligibleForPromotion =
            employee.getExperience() > 15 ? true : false;
        System.out.println("\t\t" + employee.getName() + "
            is eligible for promotion? " +
            eligibleForPromotion);
    }
    @Override
    public void visitTheElement(JuniorEmployee employee) {
        // We'll promote them if experience is greater
        // than 5 years
        boolean eligibleForPromotion =
            employee.getExperience() > 5 ? true : false;
        System.out.println("\t\t" + employee.getName() + "
            is eligible for promotion? " +
            eligibleForPromotion);
    }
}
// Client.java
class Client {
    public static void main(String[] args) {
        System.out.println("***Composite Pattern and

```

```

        Visitor Pattern Modified Demonstration.***");
Employee hodMath = formMathDept();
Employee hodCompSc = formComputerScDept();
Employee principal = formPrincipal(hodMath, hodCompSc);
printCollegeStructure(principal);
Visitor visitor = new PromotionCheckerVisitor();
System.out.println("\nChecking potential candidates
                    for the promotion:");
    visitCandidates(principal,visitor);
}
private static void visitCandidates(Employee emp,Visitor visitor) {
    emp.acceptVisitor(visitor);
    emp.getSubordinates().forEach(
        e->visitCandidates(e,visitor));
}

private static void printCollegeStructure(Employee principal) {
    // Printing the leaf-nodes and branches
    // in the same way
    // i.e. in each case, we
    // are calling printStructures() method.
    System.out.println("The details of the Principal object:");
    // Prints the complete college structure
    principal.printStructures();
}
private static Employee formComputerScDept() {
    // Three lecturers other than HOD work
    // in the Computer Science Department.
    Employee cseTeacher1 = new JuniorEmployee("C.Kate",
                                                "Lecturer", 8.2);
    Employee cseTeacher2 = new
        JuniorEmployee("C.Jones", "Lecturer", 4.5);
    Employee cseTeacher3 = new
        JuniorEmployee("C.Proctor", "Lecturer", 6);
    Employee hodCompSc = new SeniorEmployee(
        "Mr. V.Sarcar", "HOD(CSE)", 15.5);
    // Teachers of Computer Science
    // directly reports to HOD-CSE
    hodCompSc.addEmployee(cseTeacher1);
    hodCompSc.addEmployee(cseTeacher2);
    hodCompSc.addEmployee(cseTeacher3);
    return hodCompSc;
}
private static Employee formMathDept() {
    // Two lecturers other than HOD work
    // in the Mathematics department

```

```

    .

    Employee mathTeacher1 = new JuniorEmployee(
        "M.Jacob", "Lecturer", 7.5);
    Employee mathTeacher2 = new JuniorEmployee(
        "M.Rustom", "Lecturer", 3);
    Employee hodMaths = new SeniorEmployee(
        "Mrs.S.Das", "HOD(Math)", 12.5);
    // Teachers of Mathematics directly
    // reports to HOD-Mathematics
    hodMaths.addEmployee(mathTeacher1);
    hodMaths.addEmployee(mathTeacher2);
    return hodMaths;
}
private static Employee formPrincipal(Employee hod_Math,
    Employee hod_CompSc) {
    // Principal of the college
    Employee principal = new SeniorEmployee("Dr.S.Som", "Principal");
    // The Principal is on top of the college.
    // The HOD(Mathematics) and HOD(Computer Sc.)
    // directly report to him.
    principal.addEmployee(hod_Math);
    principal.addEmployee(hod_CompSc);
    return principal;
}
}

```

Output

Here's the new output. It shows that you visited the nodes in a different order and this time the principal is also eligible for promotion.

```

***Composite Pattern and Visitor Pattern Modified Demonstration.***
The details of the Principal object:
    Dr.S.Som is a Principal. Experience:21.0
    Mrs.S.Das is a HOD(Math). Experience:12.5
        M.Jacob is a Lecturer. Experience: 7.5
        M.Rustom is a Lecturer. Experience: 3.0
    Mr. V.Sarcar is a HOD(CSE). Experience:15.5
        C.Kate is a Lecturer. Experience: 8.2
        C.Jones is a Lecturer. Experience: 4.5
        C.Proctor is a Lecturer. Experience: 6.0
Checking potential candidates for the promotion:
    Dr.S.Som is eligible for promotion? true
    Mrs.S.Das is eligible for promotion? false
    M.Jacob is eligible for promotion? true
    M.Rustom is eligible for promotion? false
    Mr. V.Sarcar is eligible for promotion? true

```

```
C.Kate is eligible for promotion? true  
C.Jones is eligible for promotion? false  
C.Proctor is eligible for promotion? true
```

Q&A Session

24.1 When should you consider implementing the Visitor design pattern?

Here are some use cases to consider:

- You need to add new operations to a set of objects without changing their corresponding classes. This is the primary aim when implementing a Visitor pattern. When the operations change very often, this approach can be your savior.
- If you need to change the logic of various operations, you can simply do it through a visitor implementation.

24.2 Are there any drawbacks associated with this pattern?

Here are some drawbacks associated with this pattern:

- You can break the power of encapsulation using visitors.
- If you need to add new concrete classes to an existing architecture frequently, the visitor hierarchy becomes difficult to maintain. Suppose in demonstration 1, you want to add another concrete class in the `NumberProcessor` hierarchy now. In this case, you need to modify the visitor class hierarchy when you want to visit the newly added element.

24.3 Why are you saying that a visitor class can violate the encapsulation?

Often your visitor needs to access an object's state. To support this, you provide the getter/setter methods. Now if an object has lots of states and you expose all of them through these getters/setters, you weaken the encapsulation, because not only visitors, but other classes can use them as well.

Also, in many cases, you may see that the visitor needs to move around a composite structure to gather information and then it can modify the information. (Although in demonstration 1, I do not allow this modification). So, when you provide this kind of support, you surely violate the core aim of encapsulation.

24.4 Why does this pattern compromise encapsulation?

Here you perform some operations on a set of objects that can be heterogeneous also. And most often, your constraint is that you cannot change their corresponding classes. So, your visitor needs a way to access the members of these objects. To fulfill this requirement, you are exposing the information to the visitor.

24.5 In demonstration 2, I am seeing that in visitor interfaces, you are using the concept of method overloading. For example, you have written interface methods as follows:

```
void visitTheElement(SeniorEmployee employees);  
void visitTheElement(JuniorEmployee employee);
```

It appears to me that following the GoF book, you could use something like the following:

```
void visitSeniorEmployee(SeniorEmployee employees);  
void visitJuniorEmployee(JuniorEmployee employee);
```

Is this understanding correct?

Nice catch. Yes, you can do that if you do not like to use method overloading. For a beginner, it is often helpful, too.

But this example is simple, and I assume that you know the concept of method overloading. So, I do not see any harm in using this concept. You can simply remember that these interface methods should target the specific classes such as `SeniorEmployee` or `JuniorEmployee`.

24.6 Suppose in demonstration 1, I have added another concrete subclass of `NumberProcessor` called `RealNumberProcessor`. How should I proceed now? Should I have another specific method in the visitor interface?

Exactly. You need to define a new method that will be specific to this new class. So, your interface may look like the following (let's say you use method overloading here):

```
interface Visitor {  
    // The method to visit the IntegerProcessor.  
    void visitNumber(IntegerProcessor myInt);  
    // The method to visit the RealNumberProcessor.  
    void visitNumber(RealNumberProcessor myReal);  
}
```

```
}
```

And later you need to implement this new method in the concrete visitor classes. (I know what you are thinking! Yes, following the ISP principle, you should not make a fat interface.)

24.7 Suppose I need to support new operations in the existing architecture. How should I proceed with the Visitor pattern?

For each new operation, create a new concrete implementor (or subclass) of `Visitor` and implement the operation in it. Then visit your existing structure as shown in the preceding examples. I summarize the key steps when I discuss the implementation at the beginning of this chapter.

24.8 Is creating an object structure mandatory for me?

What is an object structure? It is like a container that helps a client to visit smoothly in one shot. So, if you deal with only one element, there is no need to create an object structure. This is why I made demonstration 1 for you.

But, consider demonstration 2, where all nodes together form a tree structure. In cases like this, you can use the Visitor pattern, too. Actually, it is a versatile pattern that can be used in different situations.

Previous chapter

[< 23. Mediator Pattern](#)

Next chapter

[25. Interpreter Pattern >](#)